# NS-3 NB-IoT Simulator User Guide

**Introduction**

This guide provides students with a fully prepared environment for running NS-3 NB-IoT simulations using Docker. The goal is to ensure a consistent, reproducible setup across all lab machines and student laptops.

The NS-3 simulator is packaged inside a Docker container, eliminating the need for manual installation of dependencies or compilation on your local system.

# 1. Overview

The NS-3 NB-IoT simulator is provided as a Docker-based environment. Docker ensures that every student runs the same version of NS-3 with identical dependencies. You will:

- Install Docker
- Build the NS-3 simulator container
- Run the container
- Execute NB-IoT simulations

# 2. Installing Docker

Docker allows applications to run inside isolated containers. Follow the official Docker installation guide:

**https://docs.docker.com/engine/install/**

If you prefer you can install its GUI (https://docs.docker.com/desktop/). In either case, choose your operating system (Windows, macOS, Linux) and follow the steps provided.

## Verify Installation

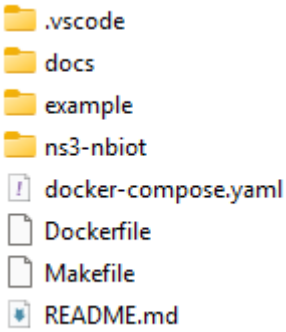
Open a terminal and run:

```
docker --version
docker compose version
```

You should see version numbers printed.

# 3. Project Structure

You must download the repository files using **git clone** [https://github.com/h3dema/ns3-simulator.git](https://github.com/h3dema/ns3-simulator.git). Inside of the folder ns3-simulator, you will see the following structure:




These files define the NS-3 simulator environment. You do not need to modify them to run the examples in this guide.

# 4. Building the NS-3 Simulator Container

Open a terminal and navigate to the project folder:

```
cd <project-folder>
```

Build the container[1]:

```
docker compose build
```

This step:

- Downloads Ubuntu 22.04
- Installs NS-3 dependencies
- Clones the NB-IoT NS-3 repository
- Builds NS-3 using waf

[1] You can also use the `make` command to manage the ns3 container. More information can be found in `README.md` in the root of the repository.

The *build* operation may take several minutes, and your computer needs to be connected to the Internet.

## 5. Running the Container

Start an interactive session inside the NS-3 environment:

```
docker compose run ns3
```

You will be placed inside the workdir defined in docker-compose.yaml. By default, it is **/opt/ns3-nbiot**. This directory contains the full NS-3 source tree.

There is **another option** to run the container and **automatically remove it from the disk when you exit it**. Use it with care, because the container is destroyed with no leftovers (you lose anything you left inside the container's disk), only the original image is intact.

```
docker compose run --rm ns3
```

## 6. Running NS-3 Simulations

Inside the container, simulations are executed using waf:

```
./waf --run <program-name>
```

For example, to run a simulation called "nb-scenario5.cc" that is in the scratch folder:

```
./waf --run scratch/nb-scenario5.cc
```

## 7. Exiting and Re-entering the Container

To exit the NS-3 environment:

```
exit
```

To re-enter later:

```
docker compose run ns3
```

## 8. Optional Cleanup

Remove the built image:

```
docker image rm ns3-simulator-ns3
```

Remove stopped containers.

```
docker container prune
```

If you run the container with the --rm option, every time you “exit” the container, it is automatically removed (and you cannot reenter it).

## 9. NetAnim

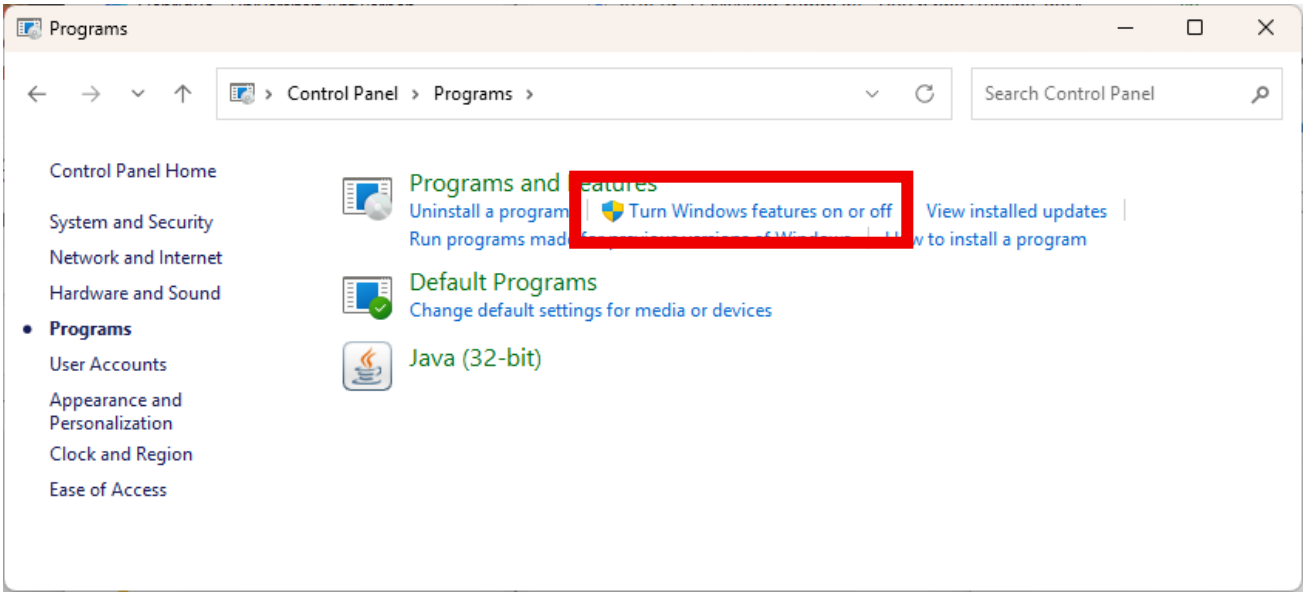
**NetAnim** is an offline animation tool built with the Qt framework. Rather than running a live simulation, it takes an **XML trace file** generated from a previously executed simulation and visually animates the network topology and traffic flow. In this guide, we are assuming that we are going to install NetAnim on the host OS. **It does not run inside the container**. The examples below are on an Ubuntu host.

### If you have a Windows 10+ computer

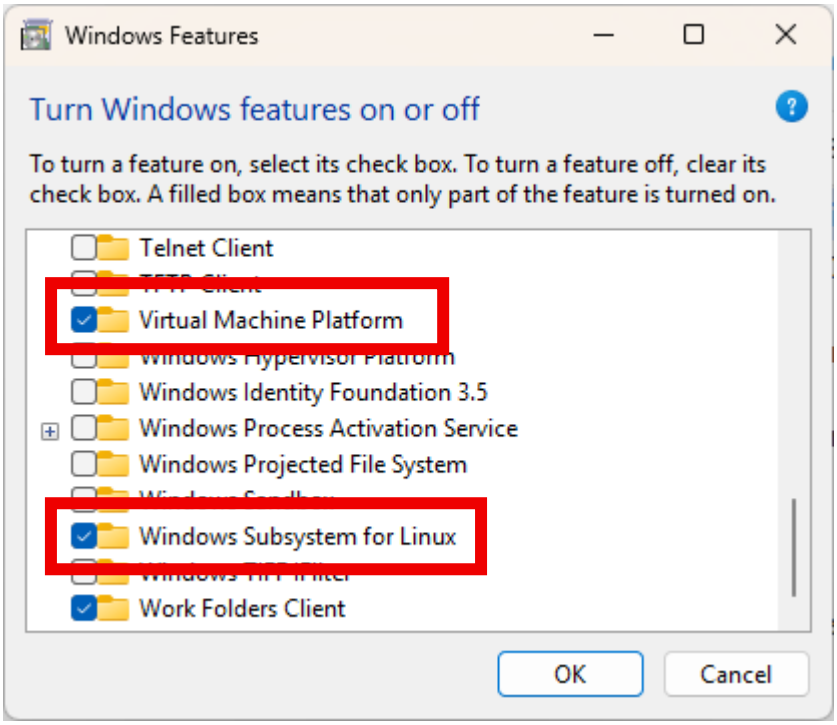
We strongly recommend using WSL (Windows Subsystem for Linux) rather than trying to build and run NetAnim natively on Windows. WSL 2 is particularly suitable because it runs a real Linux kernel while integrating with Windows. For current Windows 10 and Windows 11 systems, Microsoft provides a very simple installation procedure.

The main procedure is listed below but you can also watch a video showing the installation on YouTube, e.g., “How to Install WSL2 on Windows 11 (Windows Subsystem for Linux) | Full Guide 2026” at https://www.youtube.com/watch?v=XS6-2TXPUV4. If you are not afraid of doing all the steps typing commands on PowerShell, you can look at this other video https://www.youtube.com/watch?v=-Wg2r1lWrTc.

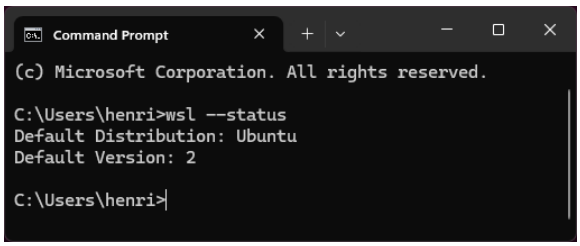
1. Open the Control Panel in your Windows computer, and select “Turn Windows features on or off” as show in the image below:

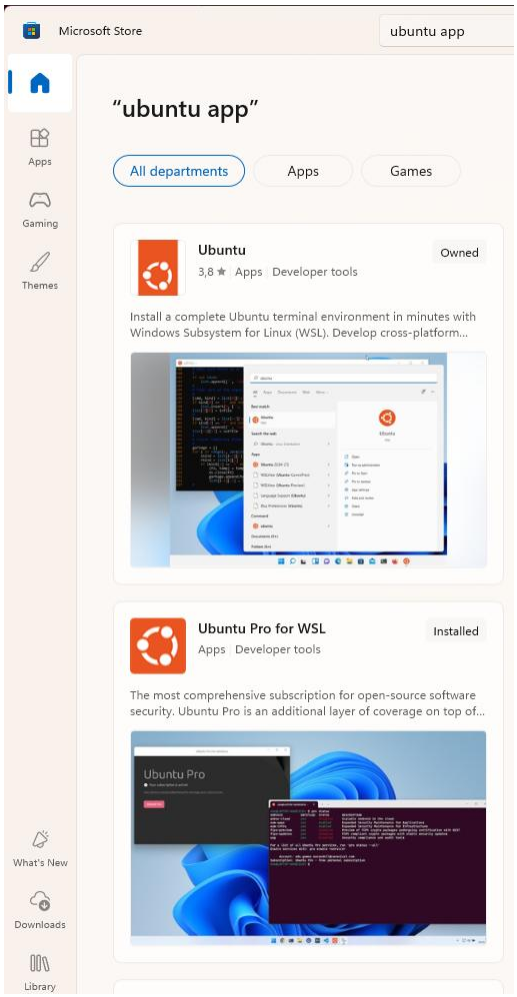



2. Inside the feature selection window, you have to select two options: Virtual Machine Platform and Windows Subsystem for Linux.



3. Click ok and select to restart your computer.
4. Open a PowerShell (or command prompt) to install. Inside the shell terminal, type **`wsl --install`**
5. It will install WSL and the base Ubuntu in one go. When it finishes, close the shell and open another one. You can know check if WSL is ok by typing **`wsl --status`**

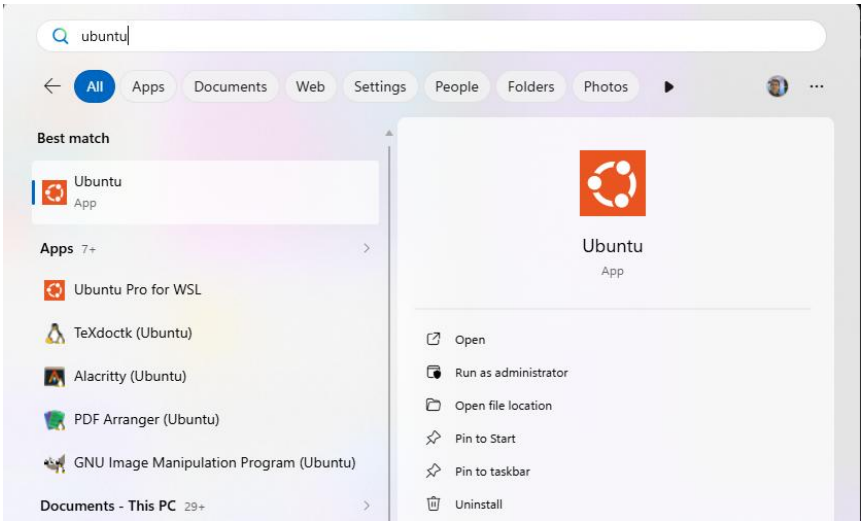



6. You still need to install the operating system to run over WSL. Open Microsoft Store, type in the search box "ubuntu app". You are going to see several options. You can use either Ubuntu or Ubuntu Pro for WSL. The image on the left shows I installed the latter but everything works the same with the other one.
To install just click on your selection, and then on the "Get" button. The distribution will be installed and, when it finishes installing, the button will become "Open".

7. The first time the Ubuntu terminal opens, your username and password are registered.

8. Later if you need, you can find the Ubuntu launcher in the start menu.




## Prerequisites & Installation on Linux

Before building NetAnim, install its required build tools and Qt5 development packages. All the commands in this section were tested in an Ubuntu 22-26 LTS desktop and in a WSL2 Ubuntu 24-26 environment. More information can be found in the documentation.

**Step 1: Install Dependencies**

Open your Linux terminal and execute:

```
sudo apt update
sudo apt install -y build-essential qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools cmake git mercurial
```

**Step 2: Download NetAnim**

Clone the official NetAnim source repository using Mercurial (hg):

```
hg clone http://code.nsnam.org/netanim
```

**Step 3: Build NetAnim**

NetAnim uses qmake (Qt's build system) for compilation and requires **Qt version 5.4 or higher**. Enter the cloned directory. Clean previous builds to ensure no stale object files remain. If it is your first compilation, just skip this step. Generate the Makefile and nuild the source code into an executable.

```
cd netanim
make clean
qmake NetAnim.pro
make
```

**Note on Compilation Warnings:** You may see several compiler warnings during the make step. **This is normal.** As long as the build finishes without an Error, your *./NetAnim* executable is ready.

## Prerequisites & Installation on MAC OS

**Step 1: Install Xcode**

```
Xcode-select --install
```

**Step 2: Install homebrew:**

```
/bin/bash -c "$(curl -fsSL
[https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh](https://r
```

**Step 3: Install cmake:**

```
brew install cmake
```

**Step 4: Install QT5:**

```
brew install qt@5
```

**Step 5: Clone the netanim repo:**

```
git clone https://gitlab.com/nsnam/netanim.git
```

**Step 6: Direct cmake to the qt5 directory:**

```
cmake -DCMAKE_PREFIX_PATH=/opt/homebrew/opt/qt@5 ..
```

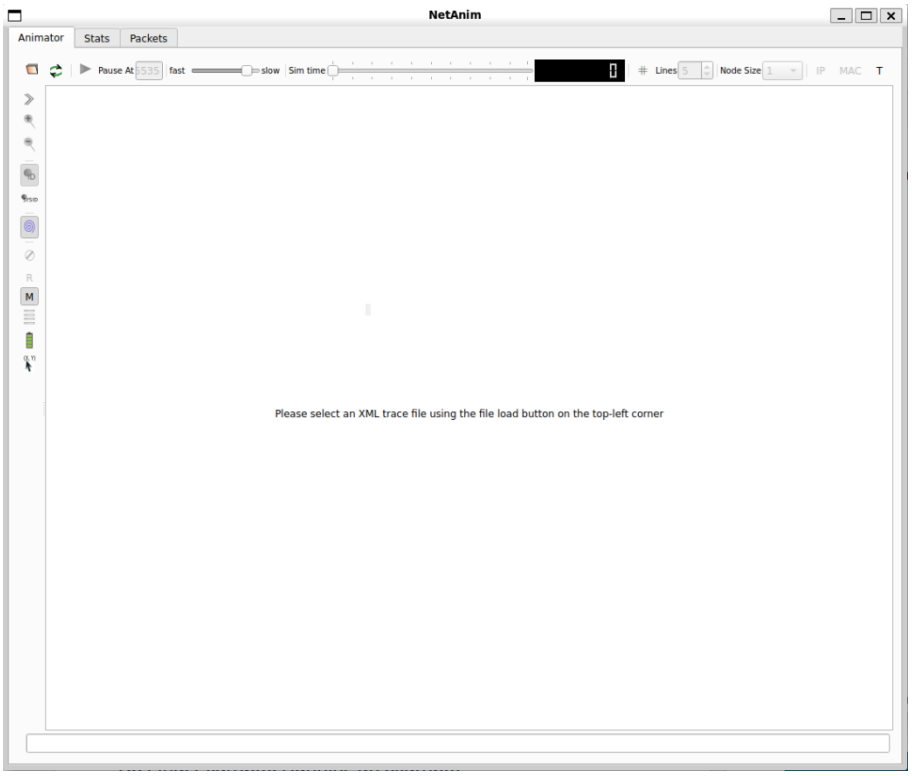
**Step 7: Build the application:**

```
cmake --build .
```

## Launch the GUI:

Execute the binary from your terminal and it will show the interface below.

```
./NetAnim
```




**1. Load the File in NetAnim**

1. Launch NetAnim using ./NetAnim.
2. Click the **Folder Icon** (top-left toolbar) to open the file browser.
3. Select and open your generated **animation.xml** file.

**2. Play the Animation**

- Click the **Play button** in the top toolbar to start playback. (Button turns green when an animation is loaded).
- Use the **Speed slider** to adjust the simulation rate or pause at specific timestamps to analyze packet transmissions.

# 10. Running an example

In this lab, you will explore energy consumption and state transitions in Cellular IoT (5G mMTC) devices using the ns-3 network simulator. You will run a simulation scenario where an IoT device equipped with an energy harvester sends small data packets over a cellular network to a remote server.

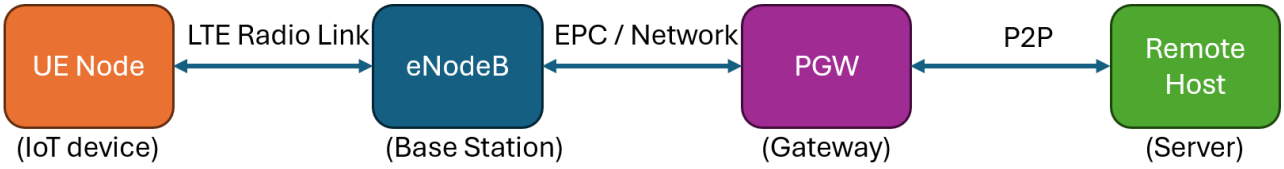
## Lab Objectives

By the end of this lab, you will be able to:

- Understand the architecture of an LTE/5G Cellular IoT simulation in ns-3.
- Run customized network simulations inside a Docker container.
- Observe state changes (ACTIVE vs. INACTIVE) driven by the energy model.
- Locate and inspect generated log files and packet traces.

## Network Scenario Architecture

The simulation (nb-scenario5.cc) models a typical IoT deployment:




1. **User Equipment (UE):** A static IoT device placed in a cell with a radius of up to 2500m. It operates on an **Energy Markov** battery model starting at 3.3 V.
2. **eNodeB (Base Station):** Fixed at the center of the cell to handle cellular coverage.
3. **Core Network & Remote Host:** Packets are routed through the LTE Evolved Packet Core (EPC) to a remote host over a high-speed link (100 Gbps).

4. **Traffic Profile:** The IoT device sends 49-byte UDP packets (simulating a 32-byte mMTC payload + CoAP/DTLS headers) every 10 seconds. Tip: Just look for UDP packets in the listing.

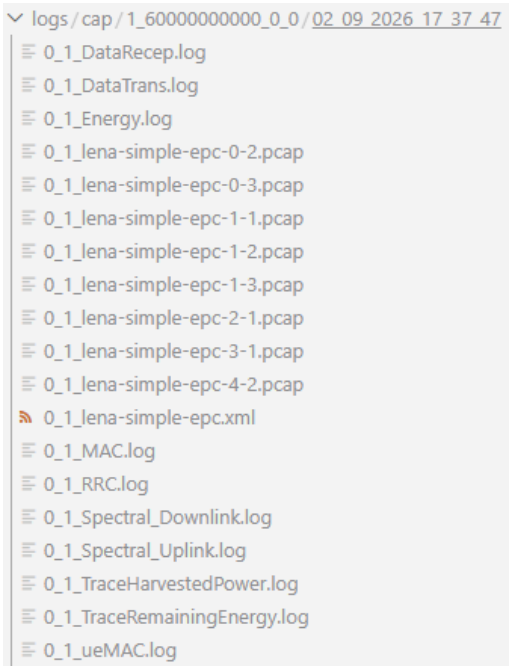
Ensure your Docker container environment is running. To prevent long simulation times during class, limit the simulation time to **60 seconds** using the --simTime parameter. By default, this parameter is set to 180 minutes. Execute the following command inside your container shell:

```
cd /opt /ns3-nbiot
./waf --run "scratch/nb-scenario5.cc --simTime=60"
```

We can define the number of Ues (i.e., IoT devices in the simulation) using the parameter –num_ues. This parameter is, by default, equal to 1. Notice that the parameters for our scenario are provided inside quotation marks (that cover the name of our cpp code inclusive). Pay attention to how each parameter is formatted: *--name_of_the_parameter=value*.

## Inspecting Simulation Outputs & Logs

Once the simulation finishes, output files and traces are automatically organized into structured log directories. Inside the container, all log files are written to “logs” folder inside the ns3 directory. *This container folder is mapped to ./logs on your local host system so you can easily view files using your local text editor or tools*. The code automatically structures outputs based on your run configuration:



**Log & Trace Output**

1. **Console Output:** You will see state transition events printed directly to standard output as the energy levels shift:

```
root@77c04d65a738:/opt/ns3-nbiot# ./waf --run "scratch/nb-scenario5.cc --simTime=60"
Waf: Entering directory `/opt/ns3-nbiot/build'
[1920/1966] Compiling scratch/nb-scenario5.cc
[1925/1966] Linking build/scratch/nb-scenario5
Waf: Leaving directory `/opt/ns3-nbiot/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (4.986s)
State ACTIVE  : 1
State INACTIVE: 0
Remote host address: 1.0.0.2
PGW address:
 * Interface #0: 127.0.0.1
 * Interface #1: 7.0.0.1
 * Interface #2: 14.0.0.5
 * Interface #3: 1.0.0.1
UE Node id: 4 Position:1377.38,493.274,1.5
UE node id: 5 IP: 7.0.0.2
Node capacitor configuration: InitialVoltage=3.3 V, Capacitance=5 F, ThresholdVoltage=1.8 V, MaxCapacitorVoltage=5.5 V
Started computation at Wed Sep  2 17:37:47 2026
Number of UEs: 1
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if it is stationary
AnimationInterface WARNING:Node:2 Does not have a mobility model. Use SetConstantPosition if it is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if it is stationary
AnimationInterface WARNING:Node:2 Does not have a mobility model. Use SetConstantPosition if it is stationary
TraceDelay TX 49 bytes to 1.0.0.2 Uid: 12 Time: +0.01s
0.01s: State changed at node 5 from 0 to 1
TraceDelay TX 49 bytes to 1.0.0.2 Uid: 13 Time: +10.01s
10.01s: State changed at node 5 from 1 to 0
30.01s: State changed at node 5 from 0 to 1
TraceDelay TX 49 bytes to 1.0.0.2 Uid: 40 Time: +40.01s
TraceDelay TX 49 bytes to 1.0.0.2 Uid: 67 Time: +50.01s
50.01s: State changed at node 5 from 1 to 0
Finished computation at Wed Sep  2 17:38:41 2026
elapsed time: 53.4635s
logs in logs/cap/1_60000000000_0_0/02_09_2026_17_37_47/0_1_
root@77c04d65a738:/opt/ns3-nbiot#
```

2. **PCAP Traces:** File captures (such as lena-simple-epc-*.pcap) are stored in the log directory. You can open these with **Wireshark** on your host machine under ./logs/ to analyze packet flows. The instructions to install wireshark can be found in https://www.wireshark.org/download.html. You can even open the files online at https://pcap.chat. Just click on “start editing”, upload the file and click on “start editing” again. Notice among the console messages that the code informs the IP addresses of the Remote Host and the UE’s. Notice that the zero time in the pcap files corresponds to approximately 10 seconds of the actual simulation (look at packetinterval_app in the scenario).

## Task 1: Knowing the scenario

1. Run the simulation for 30 minutes with 2 IoTs with default settings. Note how many state transitions occur in the console output.

2. Check the log folder under “./logs” on your local machine to verify that your new run generated a timestamped folder containing PCAP files. Open “**0_1_lena-simple-epc-3-1.pcap**” using Wireshark. Can you identify the transmitted packets?

3. Open the animation.xml file with NetAnim. Run the animation.
4. Open the "Energy.log". Plot energy vs. Time[2]. Analyze your findings.

## Task 2: Impact of Cell Size (Coverage & Distance)

- **Goal**: Measure how expanding the cell boundary alters energy consumption.
- **Execution**: Keep $UE = 1$ and vary --coverage:

```
# Small Cell (500 meters)
./waf --run "nb-scenario5 --simTime=1800 --num_ues=1 --coverage=500"

# Medium Cell (2500 meters - Default)
./waf --run "nb-scenario5 --simTime=1800  --num_ues=1 --coverage=2500"

# Large Cell (5000 meters)
./waf --run "nb-scenario5 --simTime=1800  --num_ues=1 --coverage=5000"

# Rural area (10000 meters from BS)
./waf --run "nb-scenario5 --simTime=1800  --num_ues=1 --coverage=20000"
```

- **Discussion**:
    - How does distance impact energy consumption during active transmission states?

## Task 3: Propagation Model Customization (No Recompilation)

- **Goal**: Override propagation parameters using ns3::ConfigStore command-line flags.
- **Execution**: Pass default attribute modifications dynamically:

[2] Check the python script `plot_cap_energy.py`.

```
# Line of Sight (LOS)
./waf --run "nb-scenario5 --
ns3::WinnerPlusPropagationLossModel::LineOfSight=true --simTime=3600"

# Non-Line of Sight (NLOS)
./waf --run "nb-scenario5 --
ns3::WinnerPlusPropagationLossModel::LineOfSight=false --simTime=3600"
```

**Discussion**:

- Compare packet delivery statistics (PCAP traces) under **LOS** vs **NLOS** environments.

## Conclusion

You now have a fully functional NS-3 NB-IoT simulation environment using Docker. This setup ensures consistency across all student machines and simplifies the process of running simulations during lab sessions.